

Setting up the UEFI bootloader

What we're going to do now is tell UEFI the name of the Linux binary so that the firmware can directly proceed to booting the kernel instead of going through the shell every time.

This is the easiest and fastest way to boot Linux on EFI right now. It is the only way if you want a sub-two-second boot time to the desktop on your EFI and SSD system. This is also the only way that's guaranteed to work—the UEFI spec is so vast, UEFI implementations invariably tend to have bugs. I have a Lenovo ThinkPad—supposedly the gold standard for Linux support, and it uses Phoenix firmware. I've never had much luck using gummiboot on this system, but EFISTUB works wonderfully.

There's a downside to this too. Since we're hardcoding the path to the kernel into the NVRAM, if you have a distro that changes the name of the kernel binary for every kernel upgrade, you're either going to have to recreate the entry for Linux every time, or you're going to have to copy or rename the latest kernel and *initrd* files to something that the firmware knows. Since I use Arch Linux, I don't need to bother about this—the path to the latest kernel binary remains constant. But if you're using something like Fedora or Ubuntu, you're going to have to write a script or something to automate this, and run it every time the kernel is upgraded. Doing it manually every time the kernel is updated is soon going to become a bore.

To start with, make sure you have access to the *efivars* module. If *efivars* is built as a module, *modprobe* it, as follows:

```
$: sudo modprobe efivars
```

If it's built into the kernel, you're good to go. Install the *efibootmgr* package for your system, and run the following:

```
$: sudo efibootmgr -c -d /dev/sdX -p Y -l \vmlinuz-linux-efi -L "Linux" -u "$(cat /proc/cmdline)"
```

If your ESP is */dev/sda2*, you should replace *X* in the above command with *a*, and *Y* with *1*. Adjust the path to your kernel, specified by the *-l* appropriately (note that this is relative to */boot*, not */*). You can also adjust the name of your entry, which is specified by the *-L* option.

efibootmgr should automatically set the Linux entry as the default entry. Reboot the system. You should boot straight into Linux. If you do, the bootloader has been set up. Congratulations.

You can stop reading right here if you want to stay with this set-up. But if you want to have Secure Boot, read on.

Secure Boot with self-signed certificates

There's no harm in keeping Secure Boot disabled. The rest of UEFI is awesome enough to warrant using it. However, if you want to use Secure Boot with Linux, you should install your own certificates into the firmware. That way, you end up

taking complete control of the hardware you use.

First, though, here's a primer on Secure Boot. The whole point of Secure Boot is to allow only trusted binaries to boot the system. How do you trust a binary? You use public-key cryptography, signing your binaries with a private key, the public key to which is known to the firmware.

There are three types of keys that UEFI uses:

- The **Platform Key (PK)** is usually generated by the systems manufacturer. This is analogous to the root key in the chain of trust.
- Multiple **Key-Exchange Keys (KEK)** may be signed by the Platform Key. The KEKs specify who's allowed to add or remove keys to the system. A typical system comes with multiple KEKs enrolled, one of which is Microsoft's.
- **Signature Database Keys (db)** are those that must be used to sign the actual binaries which boot in secure mode. The db keys must be signed by either the PK or one of the KEKs in the system. Microsoft mandates that two of its db keys be enrolled in any firmware, one of which is used to sign the Windows bootloader, while the other is used to sign third-party EFI binaries (like Canonical's UEFI shim loader).

Secure Boot operates in two distinct modes. Initially, the system is placed into *Setup* mode, which allows keys to be written to the firmware. On systems that have a pre-installed operating system with Secure Boot enabled, you'll have to enter this mode manually. Once a PK is written into the firmware, the system enters *User* mode, which allows only binaries signed by the db keys to be booted. If the keys that came with the system are undisturbed, the system operates in *Standard* mode, and if you enrol your own keys, the system begins to operate in *Custom* mode. Simple, right?

So this is what we're going to do:

1. Create our own PK, KEK and a db key to sign the Linux kernel.
2. Use the KEK to sign Microsoft's db keys, so that we can boot Windows 8 (and the Canonical shim) on the system (I need to do this, but you might want to skip this part if you don't plan on booting Windows 8 or 8.1 on this system).
3. Put the system into *Setup* mode, wipe down all the keys to the system and install our own keys.
4. Enable Secure Boot and profit.

Setting up the keys

Before we can begin, install the *sbsigntool* and *efitools* packages for your distribution. Fedora and Ubuntu both ship them, and Git versions are available in the AUR for Arch Linux.

We'll start by using OpenSSL to generate our keys, as follows:

```
# create a PK certificate
$: openssl req -new -x509 -newkey rsa:2048 -subj "/CN=My Private Key/" -keyout PK.key -out PK.crt -days 36500 -nodes
```